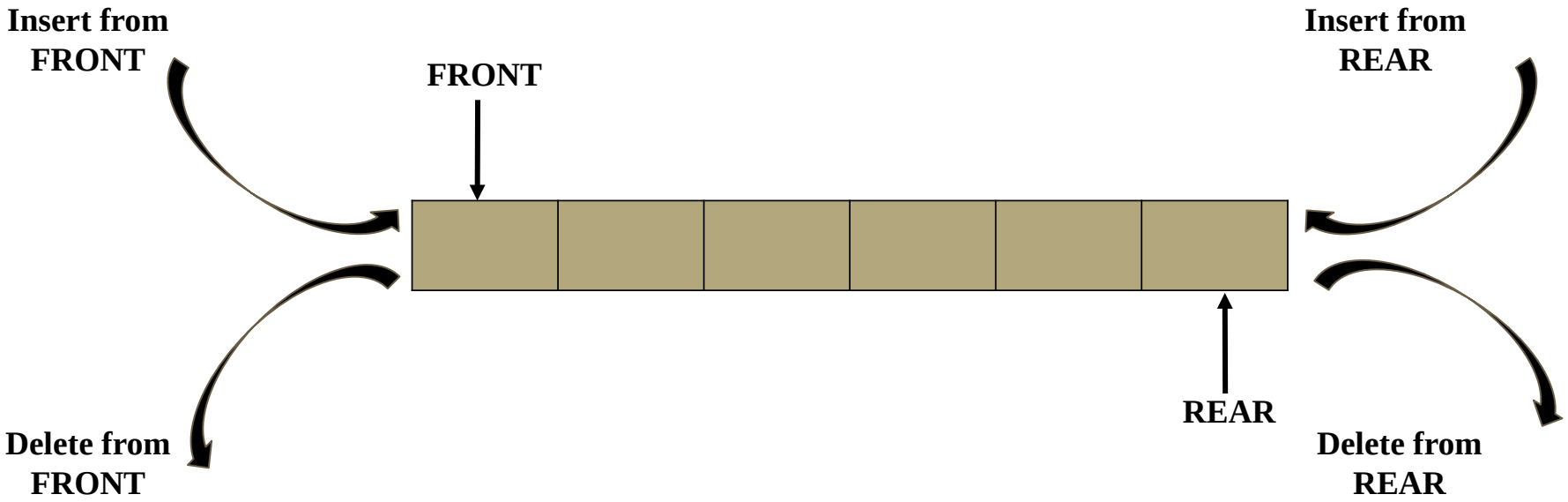


Data Structure(CSL202)

By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

Deque(double-ended queue)

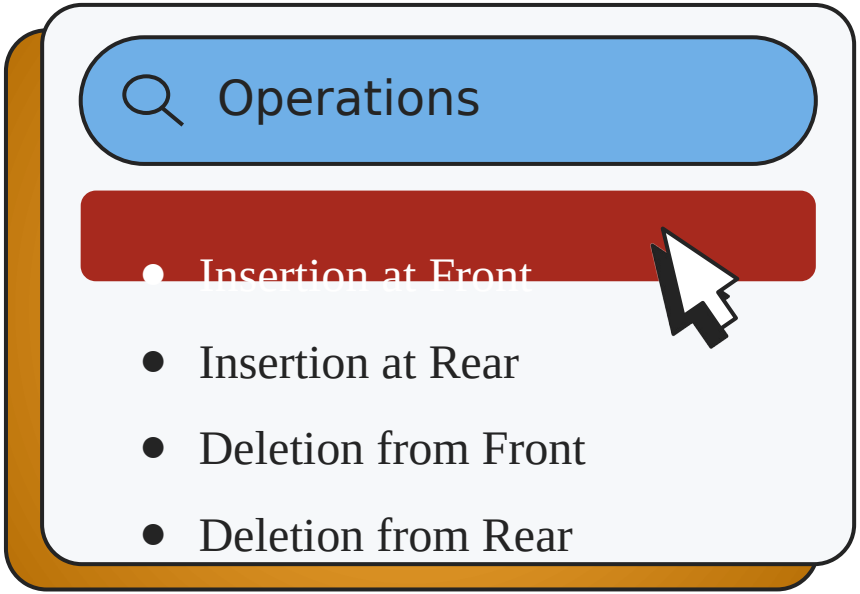
- Allows insertion and deletion from both ends —front and rear.
- Often implemented using a circular array.



Deque(double-ended queue)

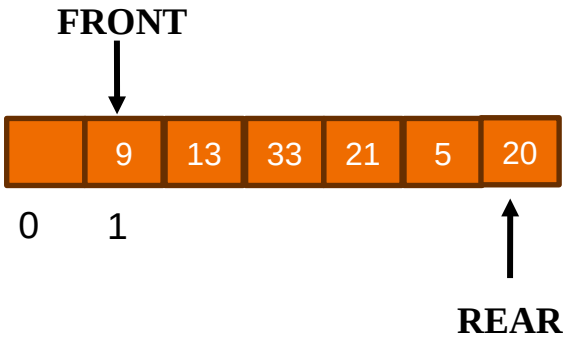
Type	Insertions Allowed At	Deletions Allowed At
Input-Restricted Deque	Rear only	Both front and rear
Output-Restricted Deque	Both front and rear	Front only
General Deque	Both front and rear	Both front and rear

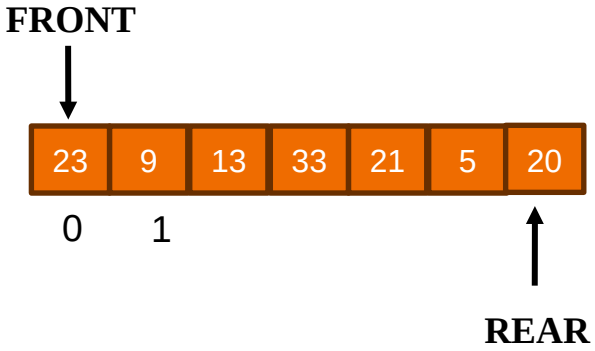
Operations in Deque



Insertion at Front

Insert 23

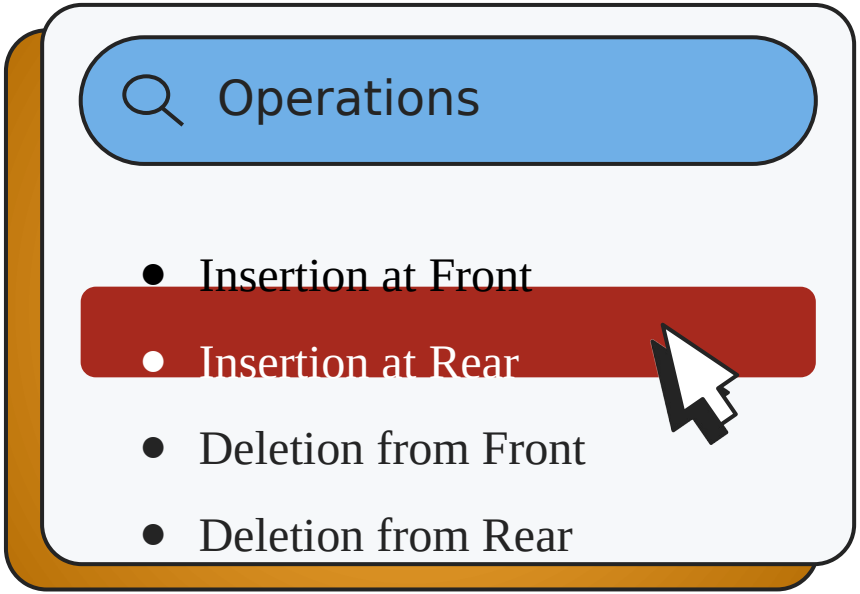




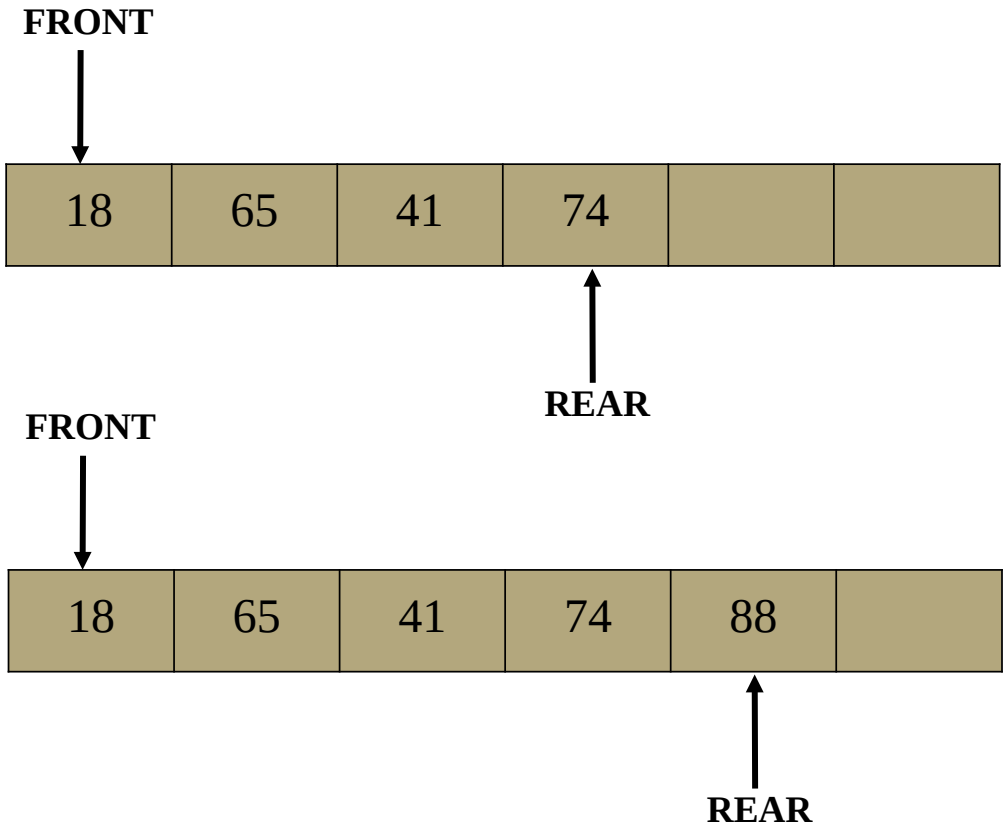
Insertion at Front

1. **[Deque Full]**
   If (FRONT == 0 and REAR == MAX-1) or (FRONT == REAR+1):
   Output "Overflow"
   Exit
2. **[Empty Deque]**
   If FRONT == -1:
   Set FRONT = REAR = 0
3. **[Wrap Around]**
   Else if FRONT == 0:
   Set FRONT = MAX - 1
4. **[Normal Case]**
   Else:
   Set FRONT = FRONT - 1
5. **Set**
   DEQUE[FRONT] = ITEM
   Exit

Operations in Deque



Insertion at Rear



Insertion at Rear

1. **[Deque Full]**

   If (FRONT == 0 and REAR == MAX-1) or (FRONT == REAR+1):

   Output "Overflow"

   Exit

2. **[Empty]**

   If FRONT == -1:

   FRONT = REAR = 0

3. **Else if REAR == MAX - 1:**

   REAR = 0

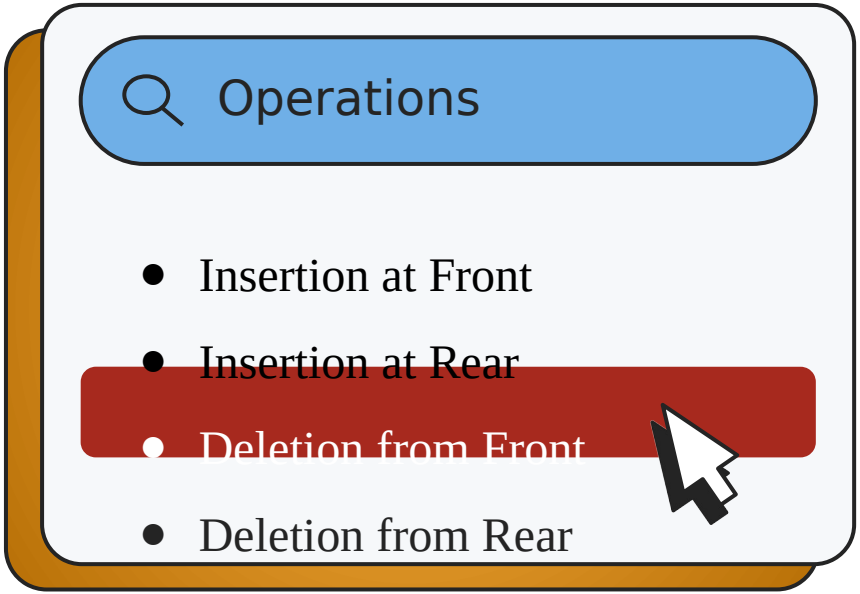
4. **Else:**

   REAR = REAR + 1

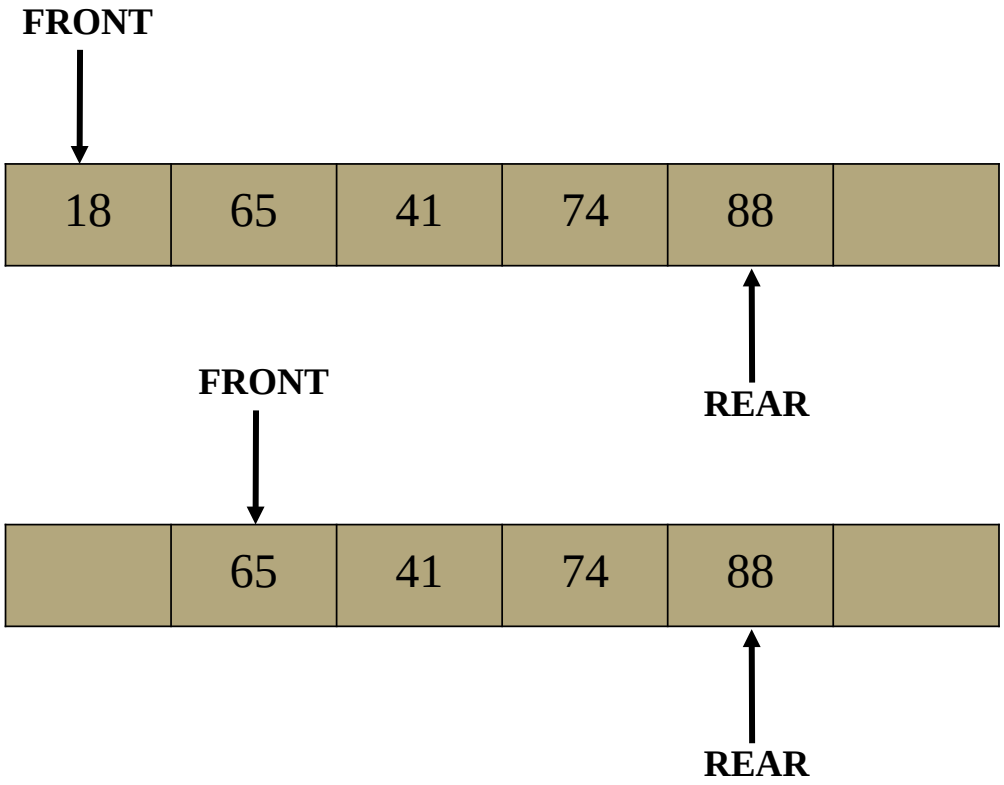
5. **Set**

   DEQUE[REAR] = ITEM

Operations in Deque



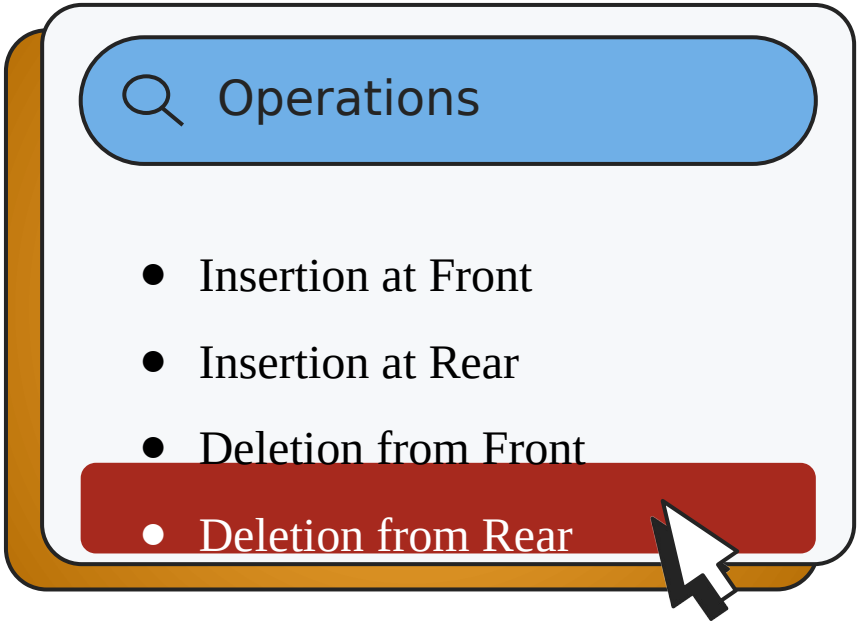
Deletion from Front



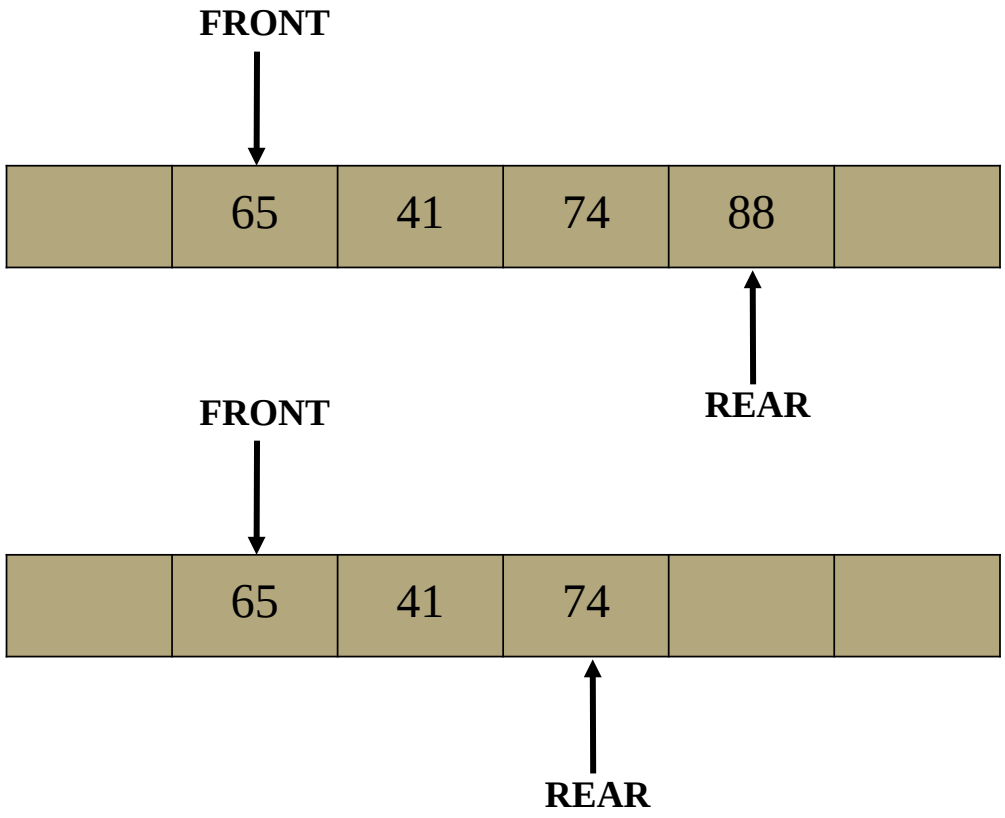
Deletion from Front

1. **[Deque Empty]**
   If FRONT == -1:
   Output "Underflow", Exit
2. **Store**
   ITEM = DEQUE[FRONT]
3. **If FRONT == REAR:**
   Set FRONT = REAR = -1
4. **Else if FRONT == MAX - 1:**
   Set FRONT = 0
5. **Else:**
   Set FRONT = FRONT + 1
6. **Return ITEM**

Operations in Deque



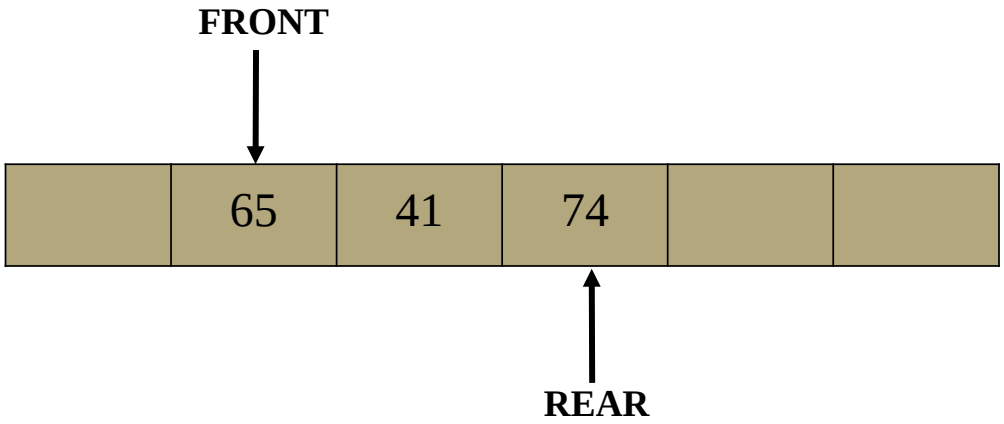
Deletion from Rear



Deletion from Rear

1. **If empty:**
   Output "Underflow", Exit
2. **Store**
   ITEM = DEQUE[REAR]
3. **If FRONT == REAR:**
   FRONT = REAR = -1
4. **Else if REAR == 0:**
   REAR = MAX - 1
5. **Else:**
   REAR = REAR - 1
6. **Return ITEM**

Deletion from Rear



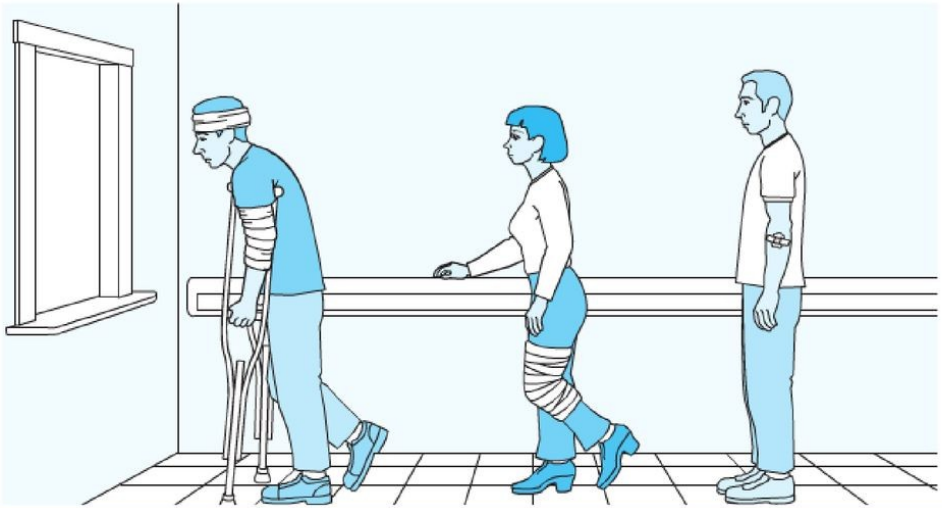
Time Complexity

- **Insert Front:** O(1)
- **Insert Rear:** O(1)
- **Delete Front:** O(1)
- **Delete Rear:** O(1)

All operations take constant time in a circular array.

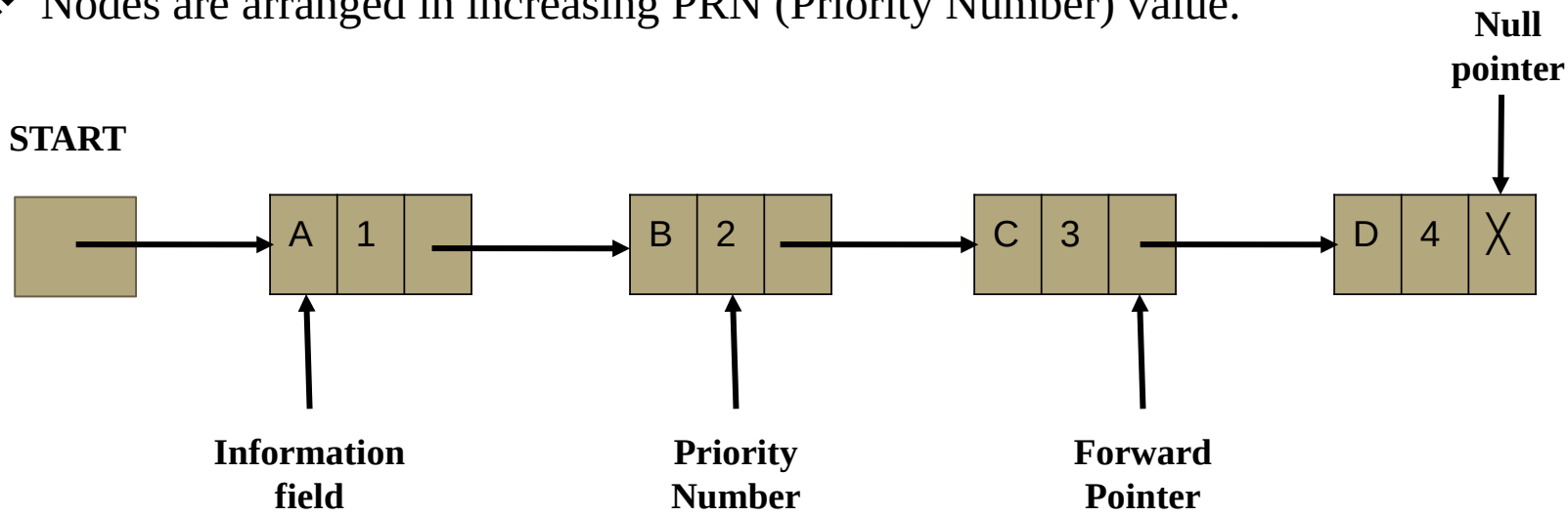
Priority Queue

- Each element has a priority.
- The element with the highest priority is served before others.
- If two elements share the same priority, they are processed in FIFO order.

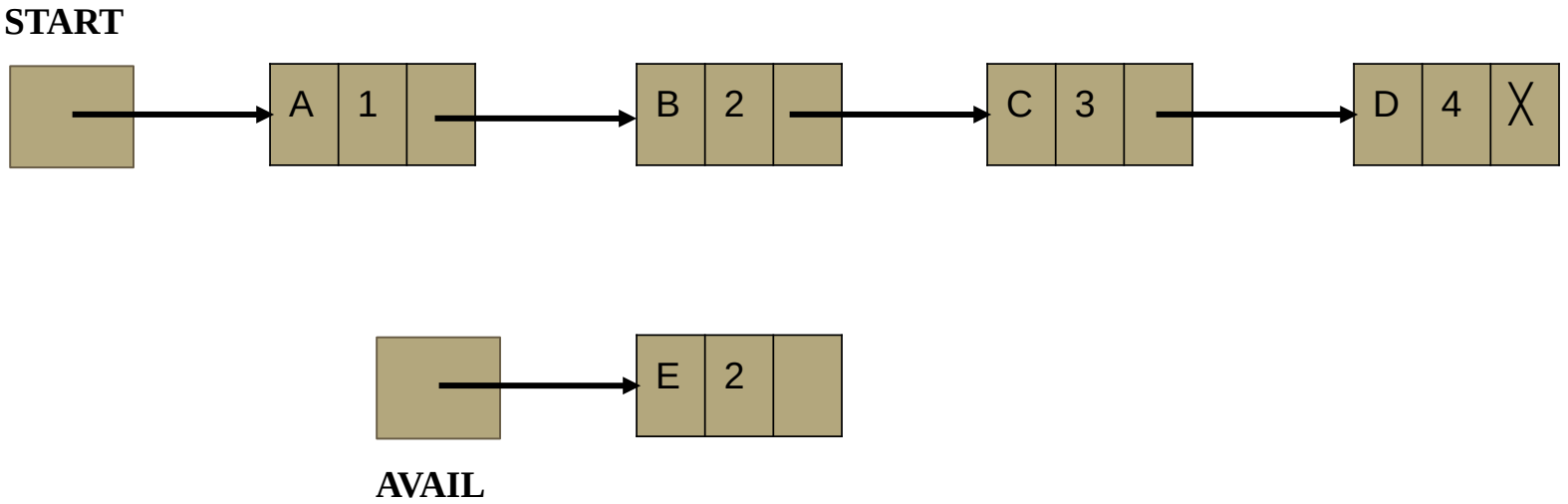


One-Way Linked List Representation

- Each node contains:
  - INFO (data)
  - PRN (priority number)
  - LINK (pointer to next node)
- Nodes are arranged in increasing PRN (Priority Number) value.



Insert an ITEM with priority number N into a Priority Queue



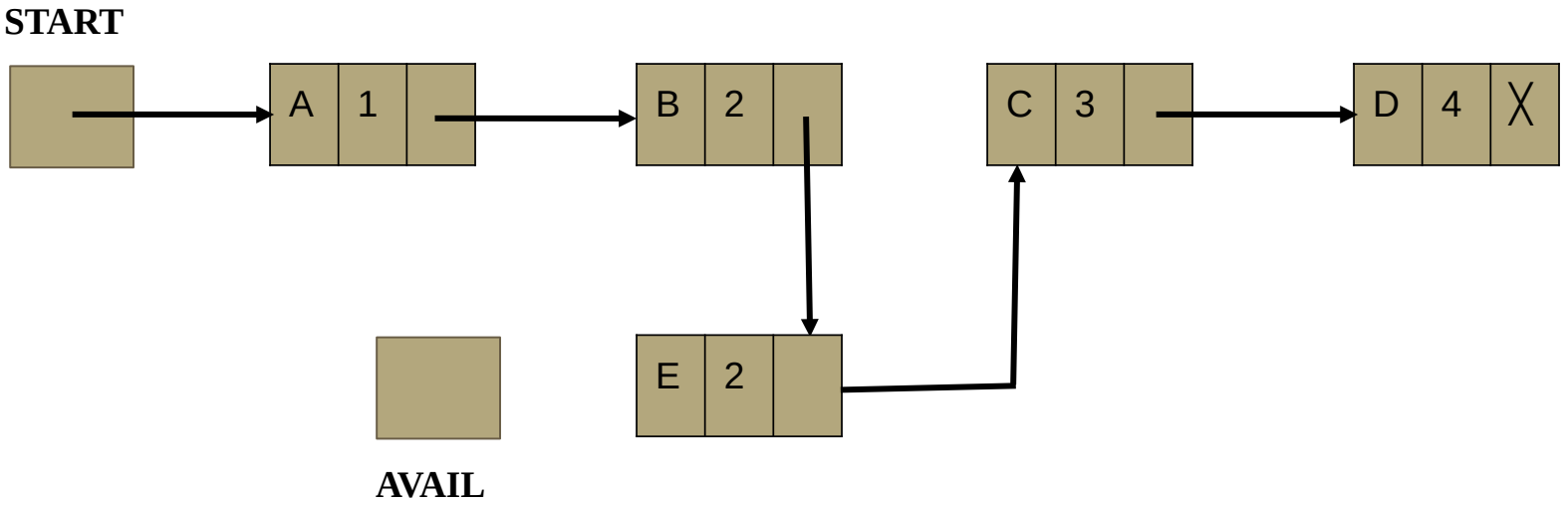
Insert an ITEM with priority number N into a Priority Queue

1. [Check for overflow]
   If AVAIL = NULL then
   Write "OVERFLOW" and Exit.
2. [Allocate node]
   NEW := AVAIL
   AVAIL := LINK[AVAIL]
   INFO[NEW] := ITEM
   PRIORITY[NEW] := N
3. If START = NULL or
   PRIORITY[START] > N then
   LINK[NEW] := START
   START := NEW

Insert an ITEM with priority number N into a Priority Queue

4. [Traverse to find insertion point]
   Set SAVE := START
   Set PTR := LINK[START]
   While PTR ≠ NULL
   and PRIORITY[PTR] ≤ N do
         SAVE := PTR
         PTR := LINK[PTR]
5. [Insert after SAVE]
      LINK[NEW] := PTR
      LINK[SAVE] := NEW
6. Exit

Insert an ITEM with priority number N into a Priority Queue

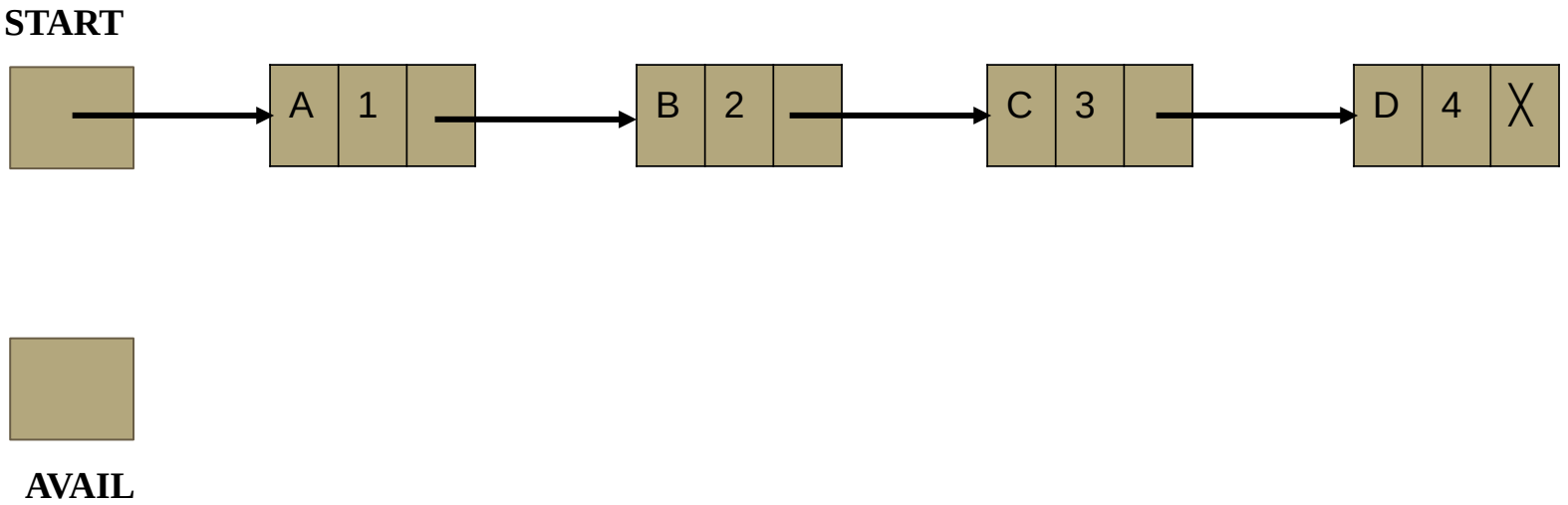


Insert an ITEM with priority number N into a Priority Queue

Time Complexity:

- Best Case: O(1) → When inserted at the beginning (highest priority).
- Worst Case: O(n) → When inserted at the end (lowest priority).
- Average Case: O(n) → Requires traversal based on priority ordering.

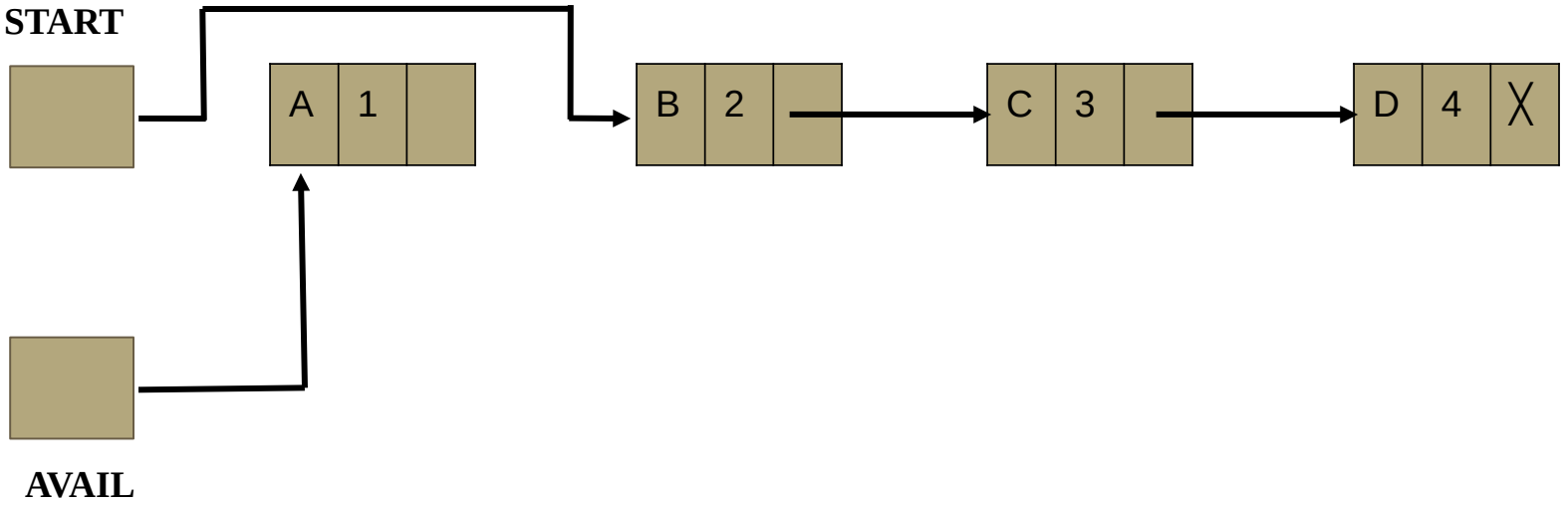
Deletes the first (highest-priority) elemen



Deletes the first (highest-priority) element

1. **[Check for underflow]**
   If START = NULL then
   Write "UNDERFLOW" and Exit
2. **[Save data in the first node]**
   ITEM := INFO[START]
3. **[Delete first node]**
   TEMP := START
   START := LINK[START] // Move START to next node
4. **[Return deleted node to AVAIL list]**
   LINK[TEMP] := AVAIL
   AVAIL := TEMP
6. Exit

Deletes the first (highest-priority) elemen



Deletes the first (highest-priority) element

Time Complexity:

- Best Case: O(1)
- Worst Case: O(1)
- Average Case: O(1)

Always deletes from the front of the list constant time.